

JavaScript Const

[Back to JS page](#)

Table of Contents

- [JavaScript Const](#)
 - [Cannot be Reassigned](#)
 - [Example 1](#)
 - [Must be Assigned](#)
 - [Example 2](#)
 - [Constant Arrays](#)
 - [Example 3](#)
 - [Constant Objects](#)
 - [Example 4](#)
 - [Block Scope](#)
 - [Example 5](#)
 - [Redeclaring](#)
 - [Example 6](#)
 - [Hoisting](#)
 - [Example 7](#)
 - [Document](#)
 - [Reference](#)

Cannot be Reassigned

A variable defined with the `const` keyword cannot be reassigned:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Cannot be Reassigned</h4>
<p id="demo"></p>

<script>
const PI = 3.141592653589793;
// PI = 3.14; // This will cause an error (commented out)
document.getElementById("demo").innerHTML = PI;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Must be Assigned

JavaScript `const` variables must be assigned a value when they are declared:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Must be Assigned</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Correct: const variables must be assigned at declaration
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo1").innerHTML = "Price1: " + price1 + ", Price2: " + price2;
document.getElementById("demo2").innerHTML = "Total: " + total;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Constant Arrays

You can change the elements of a constant array, but you can NOT reassign the array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constant Arrays</h4>
<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
// You can change an element:
cars[0] = "Toyota";
// You can add an element:
cars.push("Audi");
// But you can NOT reassign the array
// cars = ["Toyota", "Volvo", "BMW"]; // ERROR
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Constant Objects

You can change the properties of a constant object, but you can NOT reassign the object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Constant Objects</h4>
<p id="demo"></p>

<script>
const car = {type:"Fiat", model:"500", color:"white"};
// You can change a property:
car.color = "red";
// You can add a property:
car.owner = "Johnson";
// But you can NOT reassign the object
// car = {type:"Volvo", model:"EX60", color:"red"}; // ERROR
document.getElementById("demo").innerHTML = "Car: " + car.type + " " + car.model + " (" + car.co
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Block Scope

Declaring a variable with `const` is similar to `let` when it comes to **Block Scope**.

The x declared in the block is not the same as the x declared outside the block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Block Scope</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const x = 10;
// Here x is 10
{
  const x = 2;
  // Here x is 2
  document.getElementById("demo1").innerHTML = "Inside block: x = " + x;
}
// Here x is 10 (not affected by block)
document.getElementById("demo2").innerHTML = "Outside block: x = " + x;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Redeclaring

Redeclaring a variable with `const`, in another scope, or in another block, is allowed. But redeclaring in the same scope is not allowed:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Redeclaring</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
const x = 2;
// const x = 3; // Not allowed in same scope (commented out)
document.getElementById("demo1").innerHTML = "Outer x: " + x;
{
  const x = 3; // Allowed in another block scope
  document.getElementById("demo2").innerHTML = "Inner x: " + x;
}
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Hoisting

Variables defined with `const` are also hoisted to the top, but not initialized.

Using a `const` variable before it is declared will result in a `ReferenceError` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Const</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Const Hoisting</h4>
<p id="demo"></p>

<script>
try {
  // Using const variable before declaring it is not allowed (hoisting but not initialized)
  const x = 5;
  // Uncommenting the line below would cause an error:
  // console.log(y);
  // const y = 10;
  document.getElementById("demo").innerHTML = "const x: " + x + " - Variables defined with const
} catch(err) {
  document.getElementById("demo").innerHTML = "Error: " + err.message;
}
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Const](#)